Project Report

Reinforcement Learning applied to Flappy Bird

Two distinct agents: Monte Carlo Control & True Online SARSA(λ)

Léo Pommier

2025–2026

Contents

1 Introduction **2**
1.1 Environment: `TextFlappyBirdEnvSimple` 2

2 Agent 1 — Monte Carlo Control (On-Policy) **2**
2.1 Algorithm 2
2.2 Hyperparameters 3
2.3 Training Results 3
2.4 Policy and Value Function 4
2.5 Analysis 4

3 Agent 2 — True Online SARSA(λ) **5**
3.1 Algorithm 5
3.2 Linear Approximation of the Value Function 5
3.3 Hyperparameters 5
3.4 Training Results 6
3.5 Policy and Value Function 6
3.6 Analysis 7

4 Comparison of the Two Agents **8**
4.1 Summary Table 8
4.2 Policy and histogram Comparison 8
4.3 Bias–Variance Trade-off 9
4.4 Generalization to New Configurations 9

5 Code Structure **10**
5.1 Common Interface 10

6 Conclusion and Answers to Selected Questions **11**

1. Introduction

This project is part of the Reinforcement Learning (RL) course. The goal is to design and compare two agents capable of playing the game Flappy Bird in a simplified text environment, by applying two distinct families of RL algorithms.

Flappy Bird is a relevant test environment for RL: the state space is continuous by nature (distances to the obstacle), the consequences of an action are immediate (collision or survival), and the optimal policy is non-trivial yet tractable.

1.1. Environment: `TextFlappyBirdEnvSimple`

The environment is implemented as a Gymnasium (OpenAI Gym) environment. At each timestep, the agent receives an observation as a tuple (`dx`, `dy`):

- **dx**: horizontal distance between the bird and the next pipe.
- **dy**: vertical distance between the bird and the center of the gap between the pipes (**dy** is negative when the bird is above the center of the gap).

The action space is discrete and binary:

- **Action 0 — Idle**: the bird falls under gravity.
- **Action 1 — Flap**: the bird flaps its wings (moves up).

The reward is +1 at each timestep where the bird remains alive. The episode ends when the bird hits a pipe, the ground, or the ceiling.

I added a limit of 10,000 steps per game to control the duration of episodes played by our agents.

Table 1: Environment configuration parameters.

Parameter	Default value	Description
`height`	15	Grid height (in cells)
`width`	20	Grid width (in cells)
`pipe_gap`	4	Size of the opening between pipes

2. Agent 1 — Monte Carlo Control (On-Policy)

2.1. Algorithm

Monte Carlo Control is a model-free method that learns directly from complete episodes. Updates are only performed at the end of an episode, once the cumulative return G is known for each (state, action) pair.

The implemented algorithm is **First-Visit MC Control epsilon-soft**:

1. Generate a complete episode following the current ε-greedy policy.

2. Compute the Monte Carlo return:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

3. For each *first visit* to a pair (s, a), update $Q(s, a)$:

$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)} \big[G - Q(s, a)\big]$$

4. Derive the greedy policy from Q.

2.2. Hyperparameters

Table 2: Hyperparameters of Agent 1 (MC Control).

Hyperparameter	Value	Role
`episodes`	5 000	Number of training episodes
`gamma`	0.99	Discount factor
`epsilon`	$0.20 \rightarrow 0.05$	Exploration rate
`epsilon_decay`	0.999	Decay of ε per episode
`eval_episodes`	20	Episodes for greedy evaluation

2.3. Training Results

Figure 1 shows the evolution of the greedy average return. A steady improvement is observed up to episode 2,000, after which the policy converges and reaches the ceiling of 10,000.

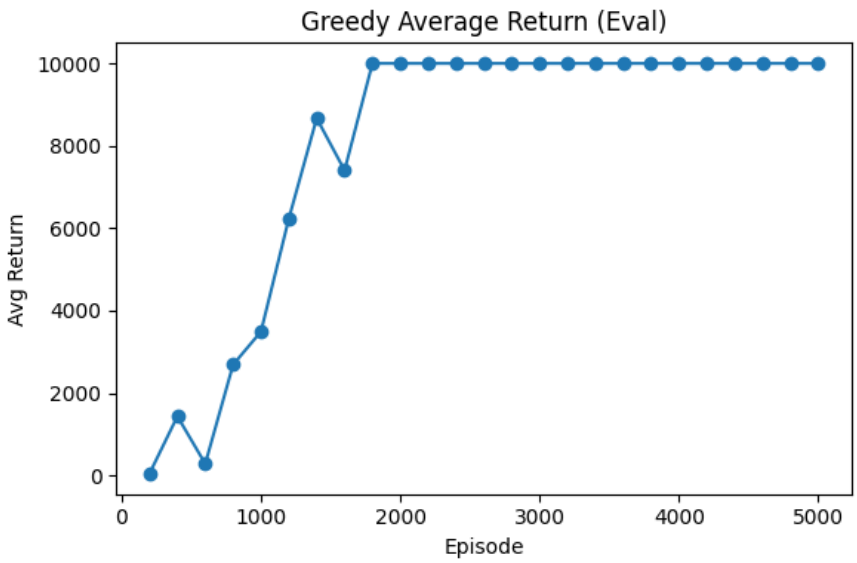
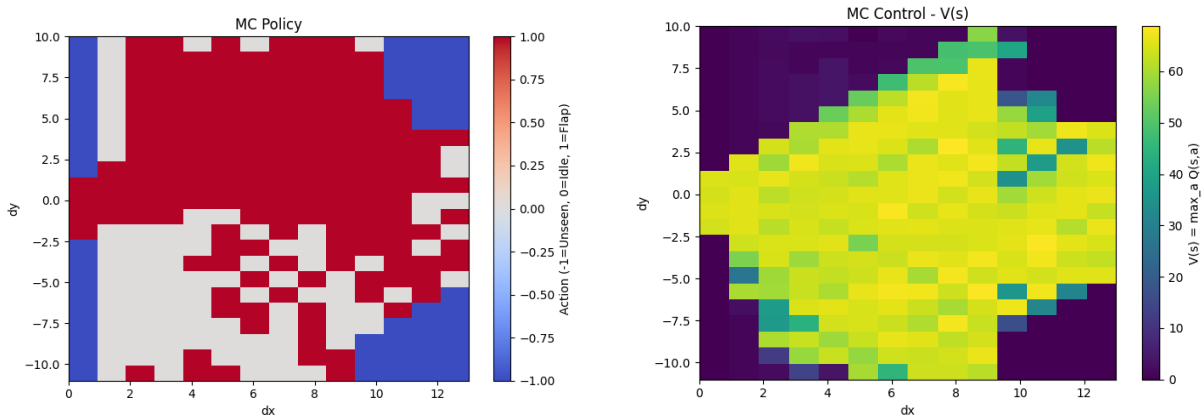


Figure 1: Learning curve of the MC Control agent.

2.4. Policy and Value Function



(a) Greedy policy (-1 = Unseen, 0 = Idle, 1 = Flap).

(b) Value function $V(s) = \max_a Q(s, a)$.

Figure 2: Policy and value function learned by the MC Control agent.

The MC policy (figure 2a) appears **fragmented**: islands of *Flap* and *Idle* decisions coexist across all regions of the space, including those where a homogeneous decision would be expected. This is characteristic of a tabular representation over a partially covered space. The blue regions allow us to identify states that are unreachable for the bird.

The value function (figure 2b) is however coherent: the highest values concentrate in the central band ($dy \in [-5, 5]$), corresponding to the states most frequently visited by a high-performing policy.

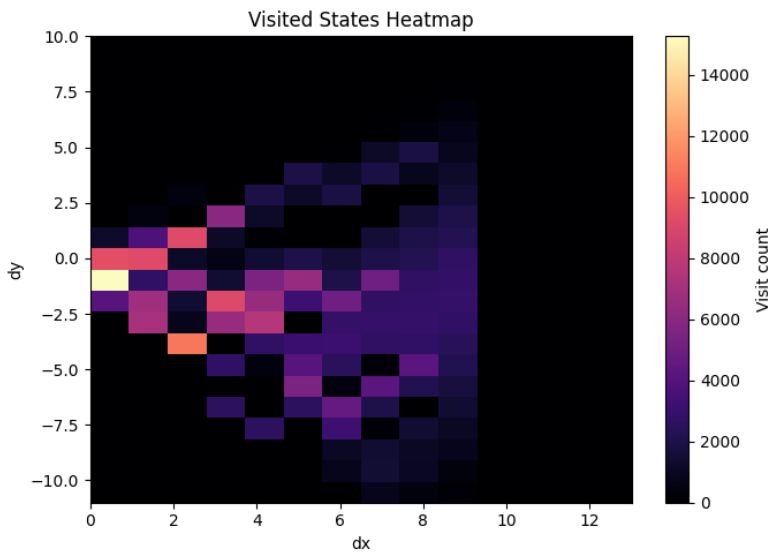


Figure 3: Histogram of visited states by the MC Control agent in greedy evaluation (200 episodes).

This histogram confirms that the central regions are the most visited.

2.5. Analysis

Advantages

- Guaranteed convergence to Q^* under sufficient coverage of the state space.
- **Unbiased** updates: returns are computed from real trajectories.
- Robustness: once converged, the policy is very stable.

Limitations

- Must wait until the end of each episode — slower learning.
- **High variance** of estimates for long episodes.
- Fragmented policy in rarely visited regions.

3. Agent 2 — True Online SARSA(λ)

3.1. Algorithm

SARSA(λ) is a temporal difference (TD) algorithm that combines **online** updates with **eligibility traces**. The implementation uses the **True Online SARSA(λ)** variant:

$$\mathbf{z} \leftarrow \gamma\lambda\,\mathbf{z} + \left(1 - \alpha\,\gamma\lambda\,\mathbf{z}^\top\mathbf{x}\right)\mathbf{x} \tag{1}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\,(\delta + Q - Q_{\text{old}})\,\mathbf{z} - \alpha\,(Q - Q_{\text{old}})\,\mathbf{x} \tag{2}$$

where $\delta = R + \gamma Q' - Q$ is the TD error.

3.2. Linear Approximation of the Value Function

The function Q is approximated by a linear model $q(s, a; \mathbf{w}) = \mathbf{w}^\top\mathbf{x}(s, a)$, with polynomial features:

$$\boldsymbol{\phi}(s) = \left[1,\ \tilde{dx},\ \tilde{dy},\ \tilde{dx}\cdot\tilde{dy},\ \tilde{dx}^2,\ \tilde{dy}^2\right]^\top \in \mathbb{R}^6$$

Action features are obtained by selective concatenation, giving $\mathbf{x}(s, a) \in \mathbb{R}^{12}$ (**12 parameters in total**).

3.3. Hyperparameters

Table 3: Hyperparameters of Agent 2 (True Online SARSA(λ)).

Hyperparameter	Value	Role
`alpha`	0.1	Learning rate
`gamma`	0.99	Discount factor
`lambda_`	0.9	Eligibility trace parameter
`epsilon`	$0.20 \to 0.05$	Exploration rate
`keep_best`	`True`	Save best snapshot

3.4. Training Results

The learning curve (figure 4) reveals a very different behavior from MC Control: performance oscillates strongly throughout training, with peaks reaching the maximum score (episodes 2,200 and 3,400) but without lasting stabilization.

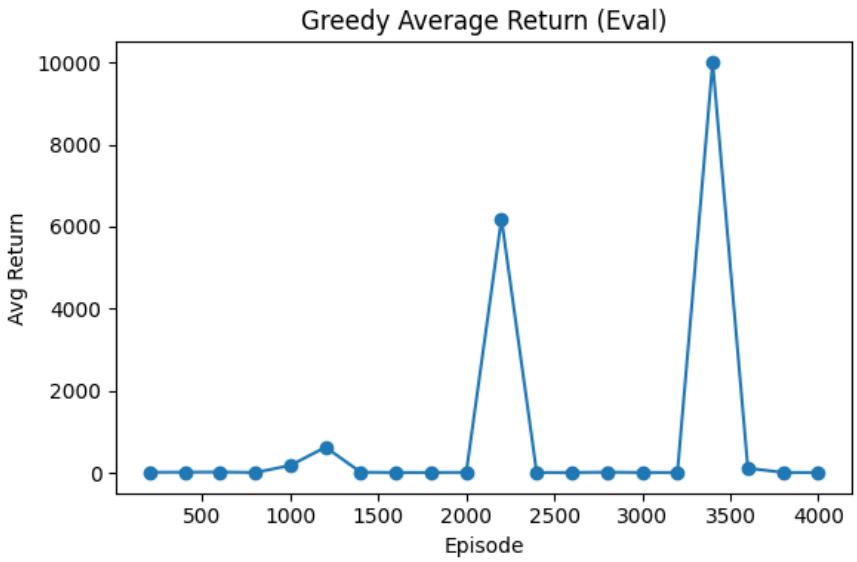
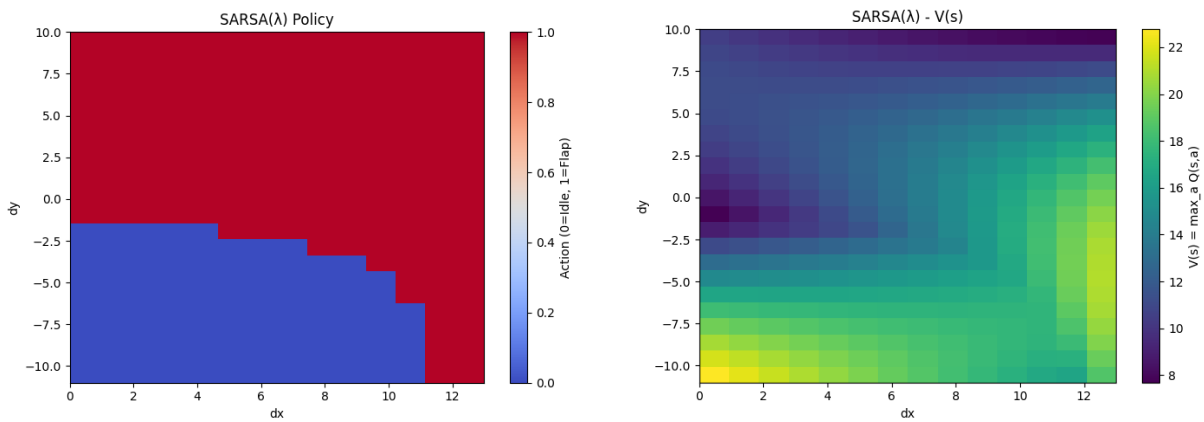


Figure 4: Learning curve of the SARSA(λ) agent.

This instability is characteristic of function approximation methods: continuous updates to $\mathbf{w}$ can degrade a good policy in important regions of the state space. The `keep_best` mechanism allows the best observed snapshot to be retained. We tried reducing `alpha` to 0.001 and `lambda` to 0.7 without any improvement in learning stability. A technique to improve learning could be to clip weight changes. Since our training already finds an optimal policy, we did not pursue further improvements.

3.5. Policy and Value Function



(a) Greedy policy (0 = Idle, 1 = Flap).

(b) Value function $V(s)$.

Figure 5: Policy and value function learned by the SARSA(λ) agent.

Unlike the MC policy, the SARSA(λ) policy (figure 5a) exhibits a **smooth and monotone decision boundary**: a direct consequence of the linear approximation, which imposes a global structure on the policy.

The value function (figure 5b) is also smooth, with a clear gradient: values increase as dx grows (being farther from the pipe implies more time to survive). This Q-value is harder to explain than that of the MC agent; it is difficult to understand why the lower-left corner has a high Q-value. A likely hypothesis is that, since the Q-value is a linear function, extremities are more easily rewarded. However, by examining the policy, we note that the lower-left corner is not a state that the policy will push the agent to visit.

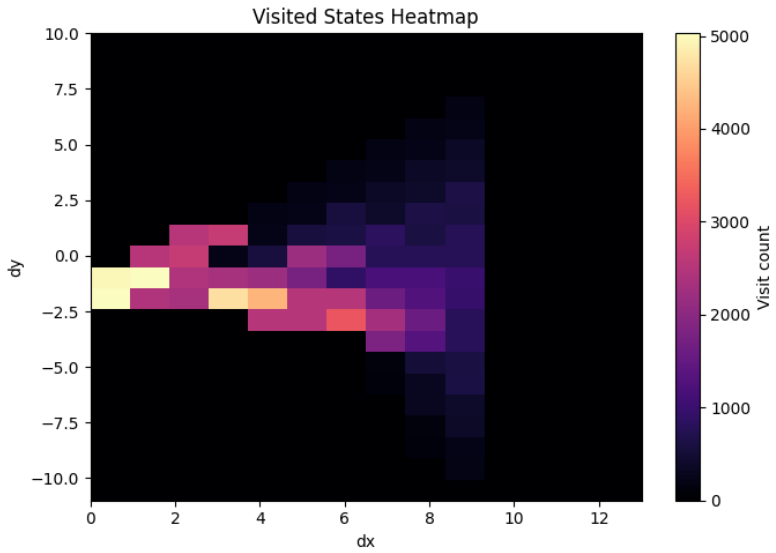


Figure 6: Histogram of visited states by the SARSA(λ) agent in greedy evaluation (200 episodes).

We can confirm that the visited states are consistent with the Flappy Bird game; the lower-left corner states are not visited despite their high Q-value.

3.6. Analysis

Advantages

- Online updates — signal propagated from the very first episode.
- Structurally regular policy thanks to linear approximation.
- Low memory footprint: 12 scalars regardless of the environment size.

Limitations

- **Training instability**: oscillations without lasting convergence.
- Bias introduced by function approximation.
- Sensitivity to hyperparameters α and λ.

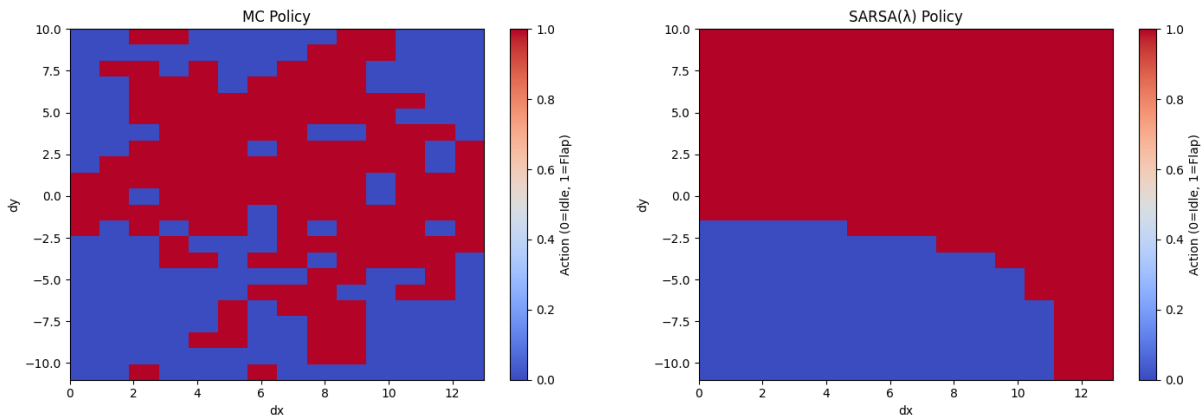
4. Comparison of the Two Agents

4.1. Summary Table

Table 4: Synthetic comparison of the two agents.

Criterion	**MC Control**	**SARSA(λ)**
Family	Monte Carlo	TD with traces
Update	End of episode	Online (each step)
Q representation	Tabular	Linear ($\mathbf{w}$)
Size of Q	$O(\|S\| \times \|A\|)$	$O(d \times \|A\|) = 12$
Bias	None	Present
Variance	High	Lower (TD)
Policy	Fragmented	Smooth, monotone
Stability	Very stable	Oscillating
Convergence	Guaranteed to Q^*	Approximation of Q^*

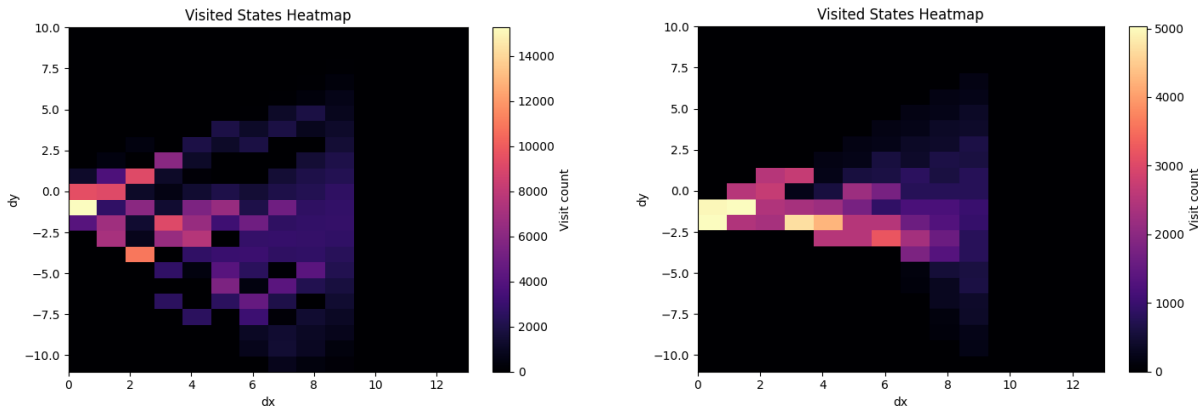
4.2. Policy and histogram Comparison



(a) Policy — MC Control.

(b) Policy — SARSA(λ).

Figure 7: Comparison of learned greedy policies. The fragmentation of MC contrasts with the smooth boundary of SARSA(λ).



(a) MC Control. (b) SARSA(λ).

Figure 8: Histogram of visited states in greedy evaluation (200 episodes).

We observe that the MC agent's policy encourages it to visit more states than the SARSA agent. Furthermore, the MC agent's histogram shows more irregularities, which is explained by the irregularity of the learned policy.

4.3. Bias–Variance Trade-off

Fundamental trade-off in RL

Monte Carlo is unbiased but has high variance: the returns G_t depend on entire trajectories, which fluctuate strongly.

SARSA(λ) reduces variance through TD updates, but introduces a bias linked to the linear approximation. The parameter λ controls the interpolation: $\lambda = 0$ gives pure TD(0) and $\lambda = 1$ approaches Monte Carlo.

4.4. Generalization to New Configurations

Figure 9 presents the performance on three configurations not seen during training.

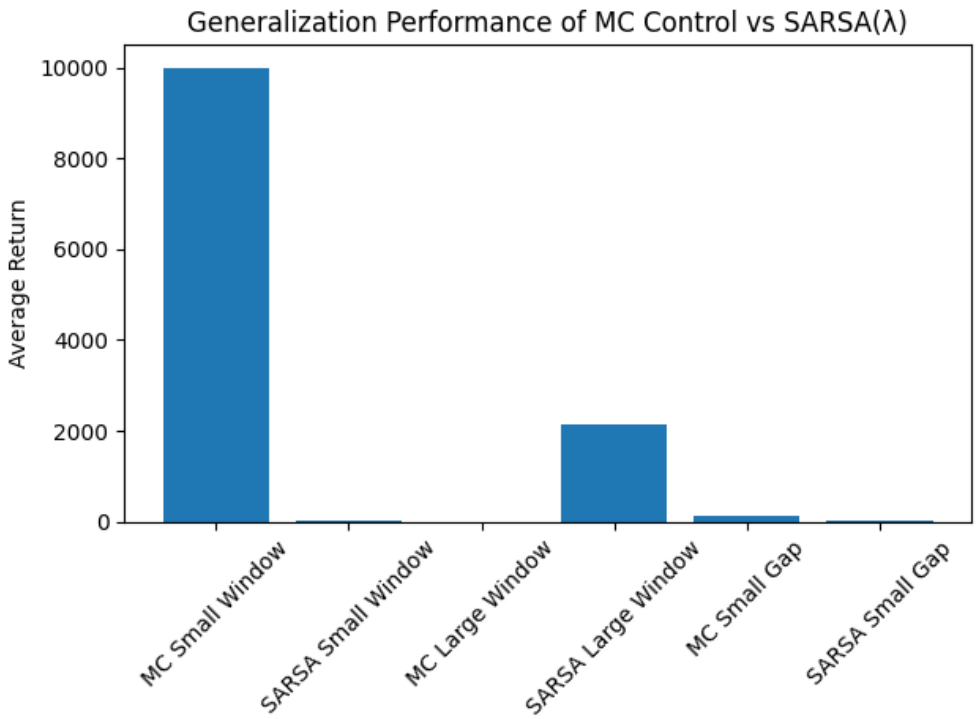


Figure 9: Generalization: average return over 100 greedy episodes on three new configurations (*Small Window*, *Large Window*, *Small Gap*).

The results are striking: MC Control largely outperforms SARSA(λ) on the configuration with a smaller window (height=10, width=15, pipe gap=4) but performs worse on the larger window (height=20, width=25, pipe gap=4), reaching 10,000 on *Small Window* and approximately 4.0 on *Large Window*, while SARSA(λ) achieves an average score of 2,000 on *Large Window*.

These results can be explained by:

- The tabular policy of MC has learned correct decisions for all visited states, which also appear in the new configurations. However, for never-visited states, the model has no idea which policy to adopt.
- The linear model of SARSA(λ) has a smooth policy and can therefore assign a coherent policy to never-visited states that are close to known ones, which explains the good performance on *Large Window*.
- The linear model of SARSA(λ) is sensitive to changes in input scale: modifying the grid size perturbs the normalized features, which would explain the poor performance of the SARSA agent on *Small Window*.

5. Code Structure

5.1. Common Interface

```
# Training
result = train(env=env, episodes=3000, gamma=0.99, ...)
```

Table 5: Description of project files.

File	Role
`text_flappy_bird_env_simple.py`	Definition of the Gymnasium environment
`train_mc_control_on_policy.py`	Agent 1: training, evaluation, visualizations
`train_sarsa_lambda_true_online.py`	Agent 2: training, evaluation, visualizations
`dev.ipynb`	Development and experimentation notebook

```
# Greedy evaluation
avg_return = evaluate_greedy(env, Q_or_w, episodes=20)

# Evaluation on arbitrary configuration
score = evaluate_on_config(Q_or_w, height=15, width=20, pipe_gap=4)

# Run agent (text rendering)
returns = run_greedy_agent(env, Q_or_w, episodes=1, render=True)

# Visualizations
plot_value_heatmap(env, Q_or_w)    # Heatmap V(s) = max_a Q(s,a)
plot_policy_heatmap(env, Q_or_w)   # Greedy policy heatmap
```

6. Conclusion and Answers to Selected Questions

This project allowed us to implement and compare two fundamental families of reinforcement learning on a concrete control problem.

The experimental results highlighted marked differences:

- **MC Control** converges stably toward the maximum score and proves logically robust to a reduction in window size; however, when the window grows, the agent is no longer able to adapt. A solution to address this issue could be to normalize the observations (dx, dy).
- **SARSA(λ)** produces a more regular policy but its training is unstable. Its generalization performance is good on a larger window thanks to its linearly estimated Q-value. However, the model performs worse when the window is reduced. We tried different learning parameters in order to stabilize training, without success. Moreover, these attempts made the policy less generalizable to window size changes.

Why choose the simple environment?

The simple environment directly encodes the important information from the screen that is useful for the bird's movement. The screen environment is heavier and does not appear to offer any concrete advantage over the simple environment.

Are our agents adaptable to the real RGB environment?

This environment is even heavier than the screen environment. A tabular model such as agent 1 could not work in such an environment, as it needs to visit all states many times.

The number of episodes required for our agent to visit all states would be far too large. On the other hand, the SARSA model uses a linear evaluation function with prior feature construction. If the feature construction is well adapted to the new environment, it would then be conceivable to use this agent.